

自然语言处理

王刚

统计与数据科学学院

2026 年春季学期

目录

第十一章 文本摘要

11.1 文本摘要概述

11.2 抽取式文本摘要

11.3 生成式文本摘要

11.4 文本摘要的评测

11.5 文本摘要语料库

第十一章 文本摘要

引言

- ▶ **文本摘要 (Text Summarization)** 是一种利用算法自动实现文本分析、内容提炼并生成摘要的技术。
- ▶ 由于互联网的快速发展，当前时代的信息量出现了指数级的增长，并远超于人类的对信息地处理和利用能力，从而导致我们无法快速地挑选和有效地运用信息，这种现象称为**信息过载 (Information Overloading)**。
而文本摘要技术可以对信息进行简化，帮助人们快速获取和理解新的信息。

11.1 文本摘要概述

文本摘要概述

- ▶ 从狭义角度来看，文本摘要的目标是为文档提炼关键信息，并构造出代表原文档中最重要或相关内容的子集（摘要）。从广义来看，除了文本形式的信息，我们还可以为图像或视频等多模态的信息进行总结，并产生以自然语言形式描述的摘要。
- ▶ 文本摘要可以追溯到 1957 年 Hans Peter Luhn 的基于统计方法的研究。随着互联网的高速发展，大量信息以复杂多样的形式存在于人们的日常生活中。文本摘要作为信息甄别和过滤的核心技术之一，受到学术界和企业界越来越多的关注。
- ▶ 以机器学习方法为代表性的抽取式文本摘要技术的研究成果不断涌现，应用落地场景日趋广泛。随着深度学习技术的不断发展，特别是基于大规模数据的预训练大模型范式的成功，文本摘要领域的研究变得更为活跃，并逐渐转向生成式摘要和实时摘要。
- ▶ 此外，随着互联网中图像、短视频等图像视频内容的不断增加，一些基于多模态信息的摘要研究也成为近年来研究热点，例如图像摘要和视频摘要等。

文本摘要发展历程

- ▶ 1958 年, Hans Peter Luhn 《The Automatic Creation of Literature Abstracts》, 揭开了通过计算机来实现自动文本摘要的序幕。
- ▶ 之后的几十年时间里, 文本摘要主要依赖于信息检索中常见的特征, 例如词频以及词频-逆文档频率 (TF-IDF) 等, 通过统计信息来进行关键信息的提取。在 1990 年之前的方法中, 文本摘要大都是通过规则和统计信息从原始文本中直接抽取 (复制) 内容, 而不是生成新的内容和抽象的摘要。
- ▶ 随着机器学习技术的快速发展, 有相当多有监督和无监督的自动摘要方法被提出来。2002 年文献提出将自动文本摘要视为一个二分类问题, 如果一个句子出现在抽取式的参考摘要中, 则认为它是“正确的”, 否则就被认为是“不正确的”。无监督方法则通过定义相关的特征进行关键信息的筛选, 从而不需要依赖人工构造的摘要。

文本摘要发展历程

- ▶ 自 2014 年开始，以深度学习和神经网络为代表的文本摘要技术逐渐兴起，它们与传统技术相比具有更加卓越的性能。2019 年 Alami 等人构建了一个基于词嵌入的文本摘要系统。其次，使用深度自编码器 (Auto-Encoder, AE) 从词频输入计算隐特征也可以为句子或单词产生表示。以深度神经网络为支撑的表示学习，为单词、句子、文档提供了高质量的语义表示，并大大促进了文本摘要的发展。
- ▶ DUC-2003 和 DUC-2004 竞赛为文本摘要任务提供了标准，由此生成式文本摘要技术迎来了快速发展。原文本和摘要都是单词序列，因此可以用序列到序列模型 (Seq2Seq) 处理输入的文本序列和输出的摘要序列。机器翻译问题与文本摘要具有明显的相似性，它们的区别在于机器翻译是信息无损的，而文本摘要是有损的。因此，一些早期的生成式文本摘要方法直接使用了机器翻译模型。早期的 Seq2Seq 模型主要是基于循环神经网络 (RNN) 结构。
- ▶ 2017 年，以 Transformer 为代表的自注意力模型凭借强大的文本建模能力和并行效率，逐渐成为了生成式文本摘要的主流。

文本摘要主要任务

- ▶ 文本摘要广泛应用于多种任务和场景中，按照任务进行划分，可以将文本摘要分为单文档摘要、多文档摘要、对话摘要、跨语言文本摘要和多模态文本摘要等。
- ▶ 单文档摘要是最基本的摘要任务，给定一篇文档作为输入，模型输出它的核心内容，且长度明显短于输入文档。根据输入文档的内容长度，可以将单文档摘要分为两类：短文本摘要和长文本摘要。
 - a) 短文本摘要中文本类型可以是新闻文本、评论文本、邮件文本等短文本。
 - b) 长文本摘要则是面向学术论文、专利文档、书籍等长文本的摘要。与短文本摘要相比，长文本的信息量更多，但文档结构更加规范化。还可以根据文档类型的不同，制定针对性的摘要策略。例如，新闻摘要关注新闻的人物、时间、地点、事件等信息；评论摘要关注产品的特性、用户的情感倾向；学术论文摘要则要更关注问题的定义、方法的描述、实验的结果等等。

文本摘要主要任务

多文档摘要目标是对同一主题下的多个文档提炼主要信息并输出摘要。

- ▶ 从应用的角度来看，对于某一新闻事件，互联网上的某个新闻单位会针对此事件进行系列报道，或者多个新闻单位对同一事件进行同时报道。对这些相关性很强的多文档提炼出一个覆盖性强、形式简短的摘要就具有重要的意义。
- ▶ 另一种应用场景是商品评论的摘要。对于某一种商品来说，商家分析其售后评论并总结出用户关注的方面和特性，有助于商家针对性地优化产品。对于多文档的学术论文摘要，我们可以通过对多篇文章的研究内容进行归纳总结，得到相关研究主题的主要挑战、主流方法和未来趋势等等。

文本摘要主要任务

对话摘要的目标是为多种类型的对话生成摘要，包括会议、通话记录、在线聊天和客服对话等场景。

- ▶ 从参与者的角度来看，对话与新闻文档最大的不同点在于对话信息涉及两个以上的参与者。除此之外，**对话摘要会涉及更多的对话语义消歧、指代消解、信息补全等问题和挑战。**
- ▶ 相比新闻摘要和论文摘要，对话的摘要很难直接从现有的内容中获取，而新闻的标题和论文的摘要可直接作为参考结果并进行大规模的摘要数据收集，因此**构建对话摘要的数据集更加困难。**
- ▶ 除此之外，对话会涉及音频、视频等信息，因此对话摘要还会涉及多模态信息的建模和理解。

文本摘要主要任务

- ▶ **跨语言文本摘要**是指输入源语言（如英文）文档，输出目标语言（如中文）摘要的任务。它的主要应用场景在于为非母语者生成摘要。
一个常见的跨语言摘要数据集是 Global Voices，它是多语言新闻的英文摘要，主要包含 15 种语言的新闻及其摘要。一般而言，跨语言文本摘要可拆分为文本摘要和机器翻译两个子任务进行处理。
- ▶ **多模态摘要**目标也是输出摘要内容，但是模型的输入和输出不仅局限于文本，还包括图像、视频、语音等多媒体数据。

例如，新闻文档中包含的图像、会议记录中的语音、电影字幕对应的视频等等。在多模态摘要中，各个模态的信息可能有重合或互相补充，如何对这些信息进行对齐、识别、筛选，做到摘要内容的不重复和不遗漏，是进行多模态文本摘要的重点。

MSMO 数据集是一个常见的多模态新闻摘要数据集，其中所包含的带有图像的新闻文档，是内容对齐的，即图像和文本描述了同样的内容。

11.2 抽取式文本摘要

抽取式文本摘要

- ▶ 抽取式文本摘要中的关键内容是从原始文本中提取的，同时所提取的内容通常不会以任何方式进行修改。关键内容包括可用于“标记”或索引文本文档的短语，或共同组成摘要的关键句（包括标题）。
- ▶ 关键文本的抽取类似于略读的过程，人们在决定是否要详细阅读一篇文档之前，会倾向于选择阅读其中的摘要（如果有）、标题、副标题、数字、文档的第一段和最后一段，以及段落中的第一句和最后一句。
- ▶ 常见的抽取式文本摘要方法可分为两大类：基于排序的方法和基于序列标注的方法。

基于排序的方法

基于排序的抽取式文本摘要的基本思想是：

- ▶ 对于一篇文档，首先按照一定的规则将其拆分为多个语义单元（如短语、句子、段落等）
- ▶ 通过某种重要性评估方法为每个语义单元进行评分。
- ▶ 按照重要性分数的大小将语义单元进行排序，最后选取分值较高的语义单元作为关键内容并组成摘要。
- ▶ 一般而言，会选取句子作为内容提取的基本语义单元，而如何衡量句子的重要性以及如何设计合适的评分方法，则是影响排序效果好坏的关键。

基于排序的方法 - Lead-3 算法

- ▶ 对于大部分类型的文档，句子在文档中的位置是一个重要的判断依据。一般而言，文档的起始句段会进行总述，结尾句段会进行总结。因此，**最简单的启发式排序方法就是按照句子的位置选择关键句作为摘要。**
- ▶ **Lead-3 算法会选取文档的前三句内容作为摘要。**这种方法虽然简单直接，但在某些场景中（如新闻文本摘要）非常有效。因此，Lead-3 常常作为一个基线方法，并将其视为文本摘要方法的性能下界。

基于排序的方法 – TextRank 算法

TextRank 是一个经典的基于排序的抽取式摘要方法，它的核心思想来自 PageRank 算法。

- ▶ PageRank 是一个基于图的排序算法，用于将网页进行排序，目的是尽可能地将重要的网页排在靠前的位置。这一类基于图的排序算法，可以递归地从图中提取全局信息，并根据节点间的某种关系来衡量节点的重要性，为节点评分排序。
- ▶ PageRank 算法将网页作为有向图的节点，将网页之间可跳转的超链接作为图的有向边。当页面 A 能跳转到页面 B 时，我们可以将其视为页面 A 为页面 B “投票”。一个页面获得的票数越多，该页面的重要性就越高。
- ▶ PageRank 模型会根据投票页面的重要性，决定其投票的权重。因此，一个页面的重要性得分取决于获得的票数，以及为其投票的节点分数。PageRank 模型节点重要性迭代计算公式如下：

$$S(V_i) = (1 - d) + d * \sum_{j \in \text{In}(V_i)} \frac{1}{|\text{Out}(V_j)|} S(V_j)$$

基于排序的方法 – TextRank 算法

- ▶ 受到 PageRank 算法的启发, 在进行文本摘要任务时, TextRank 算法将文档中的句子作为节点, 以句子间的相似度作为边, 通过迭代更新节点的重要性分数。句子之间相似度表示计算公式如下:

$$\text{Sim}(S_i, S_j) = \frac{|\{w_k | w_k \in S_i, w_k \in S_j\}|}{\log(|S_i|) + \log(|S_j|)}$$

其中, S 和 w 分别代表句子和句子中的单词, $|S|$ 代表句子 S 中的单词数量, 分子部分表示同属于两句的单词数量。相应的, 对于句子节点的评分, 采用如下公式:

$$S(V_i) = (1 - d) + d * \sum_{V_j \in \text{In}(V_i)} \frac{\text{Sim}(V_j, V_i)}{\sum_{V_k \in \text{Out}(V_j)} \text{Sim}(V_j, V_k)} S(V_j)$$

使用边的权值, 迭代更新节点的重要性分数直到收敛, 最后选取 N 个得分最高的节点句子, 作为摘要进行输出。

基于排序的方法 - MMR 算法

- ▶ **Maximal Marginal Relevance (MMR)** 也是一种基于排序的抽取式文本摘要算法。与 TextRank 算法相比，MMR 算法可以得到更加多样化的抽取结果，在保证重要性的同时避免抽取过多相似的句子。

例如，以下是一系列描述上海市的短句：“十分现代化”、“有浓厚的现代化气息”、“现代化中国的代表城市”、“广阔的发展前景”。

前三个短句虽然与上海市的相关性很大，但是都表示上海的现代化程度。如果将其作为摘要，就会出现冗余信息。MMR 算法旨在解决这样的问题，在重要性和多样性之间达到平衡。

基于排序的方法 - MMR 算法

- ▶ MMR 算法在计算句子之间的相似度以及句子与整篇文档的相似度基础上，利用如下公式**计算每个句子的分数**：

$$\text{MMR} \stackrel{\text{def}}{=} \arg \max_{V_i \in R \setminus S} [\lambda (\text{Sim}_1(V_i, Q) - (1 - \lambda) \max_{V_j \in S} \text{Sim}_2(V_i, V_j))]$$

Q 代表整篇文档（或文档类别描述）；

R 为文档中的句子集合；

S 为已被选出作为结果的句子集合，是 R 的子集；

$R \setminus S$ 代表 R 中剩余未被选中的句子集合；

$\text{Sim}_1(V, Q)$ 表示文档中某个句子和整篇文档（或文档描述）的相似度；

$\text{Sim}_2(V_i, V_j)$ 表示两个句子之间的相似度；

λ 为范围 $[0, 1]$ 内的常数，**用于调整结果的多样性， λ 越小表示算法更倾向于保证抽取结果的多样性。**

基于排序的方法 - 基于朴素贝叶斯方法的文本摘要算法

- 除了人工设定规则的方法外，也可以使用统计机器学习方法结合特征工程来进行句子抽取。一个常见的方法是使用**朴素贝叶斯模型**，根据人工构建特征，以及预先标注的训练语料完成句子抽取。

常见的句子级别用于抽取式摘要的**文本特征**包括：

- 长度**：句子的长度一定程度上可以反映信息量的大小。当句子超过或短于一定长度时，通常是不适合作为摘要的。
- 位置**：基本上所有类型的文本都有其约定俗成的写作规范。句子的内容和其位置在一定程度上是有关联的。例如，文章结尾通常包含整篇文章的总结综述。
- 关键词**：当某个句子包含该类型文本的关键词，那么这个句子很有可能是摘要的一部分。
- 提示语**：在某些类型的文档中，关键的概括和总述性语句前会有明显的提示语，例如，“综上所述”，“To conclude”一类的短语之后会紧跟重要的句子。

基于排序的方法 – 基于朴素贝叶斯方法的文本摘要算法

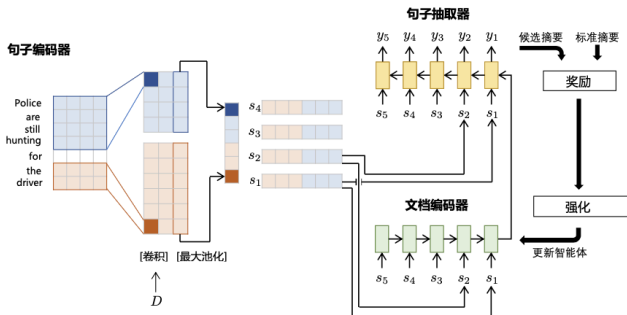
- ▶ 根据预先进行人工标注，标明句子是否为摘要句的训练集合上，根据设计好的特征值集合和标签，就可以训练朴素贝叶斯分类器，并用它来计算每个句子属于摘要的概率：

$$P(\text{label} | f_1, f_2, f_3, \dots, f_n)$$

通常需要人为设定一个阈值，当概率大于该阈值时表示当前句子是摘要的一部分，反之则不是。

基于排序的方法 - REFRESH 算法

- ▶ REFRESH 通过强化学习对抽取的句子集合进行全局优化的方法。
- ▶ 在训练阶段，每当模型抽取出若干句子，可以将此句子集合与标准摘要进行比较，计算出两者的相关性分数（一般使用 ROUGE 分数作为度量手段）作为奖励分数，通过强化学习训练策略来对模型参数进行优化更新。REFRESH 模型主要由三部分组成：句子编码器、文档编码器和句子抽取器。



基于排序的方法 - REFRESH 算法

- ▶ 句子编码器使用 CNN 中经典的时域卷积网络 (Temporal Convolutional Network) 将句子编码成连续型向量表示, $f \in \mathbb{R}^{k-h+1}$ 为其得到的时域长度, 其中 h 为卷积窗口大小, k 为句子长度, 并采用最大池化将最大值作为最后的特征保留。之后, 句子向量经过文档编码器来融合上下文中其他句子的信息。这里, 文档编码采用的是 LSTM 长短记忆循环神经网络。
- ▶ 对于核心的句子抽取器, REFRESH 使用了 LSTM 模型外加 Softmax 函数来计算似然概率 $P(y_i|s_i, D, \theta)$ 。不过, REFRESH 采用的训练策略并非极大化似然函数, 而是通过其概率分布 $P(y_i|s_i, D, \theta)$ 为文档 D 中的每一个句子 s_i 进行评分。可以选出分数最高的 n 个句子组成摘要, 并将此句子集合记为 $\hat{y}$, 将 $\hat{y}$ 与标准摘要的相关性分数作为奖励函数 $r(\hat{y})$, 则强化学习的期望奖励和损失函数可用如下式子表示:

$$\mathcal{L}(\theta) = -\mathbb{E}_{\hat{y} \sim p_{\theta}}[r(\hat{y})]$$

$$\nabla \mathcal{L}(\theta) = -\mathbb{E}_{\hat{y} \sim p_{\theta}}[r(\hat{y}) \nabla \log(\hat{y}|D, \theta)]$$

基于排序的方法 - REFRESH 算法

- 上式计算的是与句子集合 $\hat{y}$ 相关的期望项。然而，穷举所有可能的句子组合方式是不可行的。

在实际的训练过程中，REFRESH 每次从 p_θ 分布中采样单个样本 $\hat{y}$ 进行训练，来近似整个训练批次的奖励期望：

$$\nabla \mathcal{L}(\theta) \approx -r(\hat{y}) \nabla \log(\hat{y} | D, \theta) \approx -r(\hat{y}) \sum_{i=1}^n \nabla \log(\hat{y}_i | s_i, D, \theta)$$

基于排序的方法 - NeuSum 算法

- NeuSum 采用了一种将句子评分和句子选择相结合的方式来进行文档摘要。它的训练目标即为学习一个评分函数 g ，该函数可以计算某个句子被选为摘要句后，摘要整体的分数增益。在测试阶段，模型在每一个时间步 t 选出能使评分函数 g 结果最高的句子。

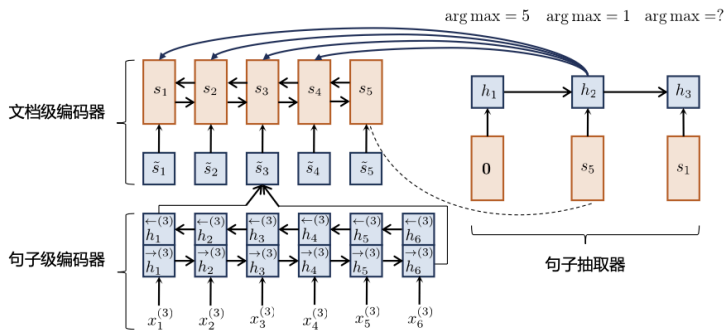
评分函数 g 如下所示：

$$g(S_i) = r(S_{t-1} \cup \{S_i\}) - r(S_{t-1})$$

上式中，NeuSum 采用了与 REFRESH 相同的评价函数 $r(\cdot)$ 来计算句子集合整体与标准摘要之间的相关性分数。 S_i 表示文档中的某个句子， S_t 表示在 t 时刻已经被选择的摘要句组成的集合。在每一个时间步 t ，模型会选出能使摘要整体增益最大的句子（即函数 g 达到最大值的句子）直至达到摘要限制的长度。

基于排序的方法 - NeuSum 算法

- NeuSum 由两个模块组成：句子编码器和句子抽取器。



NeuSum 算法中句子编码器是一个句子-文档级层次编码器。对于文档 $D = (S_1, S_2, \dots, S_L)$ (S_i 为文档中的句子)，模型先将句子 S_i 输入到一个双向的 GRU 网络中，得到句子级向量表示 $\tilde{s}_i$ ，再将得到的句子级向量输入到另一个双向 GRU 网络，可以得到融合上下文信息的句子向量 s_i 。

基于排序的方法 - NeuSum 算法

- ▶ **句子抽取器**，可以使用如下的公式计算句子 S_i 的分数，并取分数最高的句子作为 t 时刻抽取的摘要句：

$$\delta(S_i) = W_s \tanh(W_q h_t + W_d S_i)$$

基于句子分数 $\delta(S_i)$, $i \in [1, L]$ (L 为文档中的句子数量)，可以计算模型的预测分布 P ，如下所示：

$$P(\hat{S}_t = S_i) = \frac{\exp(\delta(S_i))}{\sum_{k=1}^L \exp(\delta(S_k))}$$

为了近似评分函数 g ，**NeuSum** 计算了用于训练的标签分布 Q ，并使用 **KL 散度**来缩小模型预测分布 P 和训练标签分布 Q 之间的距离。

$$Q(S_i) = \frac{\exp(\tau g(S_i))}{\sum_{k=1}^L \exp(\tau g(S_k))}$$

$$J = D_{KL}(P||Q)$$

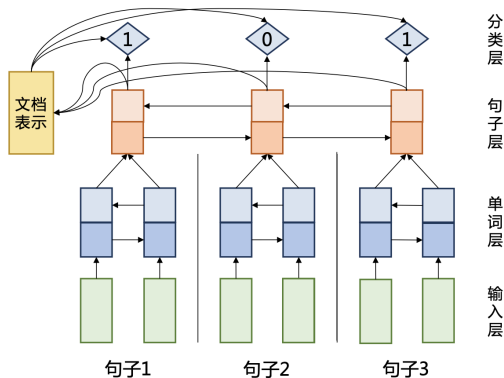
τ 为调整分布方差的温度系数。对于 $r(\cdot)$ 一般使用 **ROUGE** 分数来衡量摘要句集与标准摘要之间的相关度。

基于序列标注的方法

- ▶ 抽取式文本摘要的另一种方法是将句子抽取问题转化为序列标注任务。
- ▶ 首先要将文档分割成独立的语义单元，并将其组成语义单元序列。算法的目标就是对每个语义单元给出一个标签，来表明该部分是否要被提取出来组成摘要。一般来说，会选取句子作为基本语义单元，并使用二元标签（0 和 1）来表明句子被抽取的状态。
 - “1”表示该句子可以需要被抽取出来作为摘要句
 - “0”表示该句子不需要作为摘要句
- ▶ 与排序式文本摘要的不同点在于，基于序列标注的方法直接对句子进行二元分类，而不需要预先设计并计算句子的重要性分数，所以可以直接使用有监督学习的方法进行训练。

基于序列标注的方法 - SummaRuNNer 算法

- SummaRuNNer 算法就是一种基于 RNN 的抽取式摘要模型，该模型是一个采用两层基于门控循环单元（Gated Recurrent Unit, GRU）的双向循环神经网络，其模型结构主要包含输入层、单词层、句子层以及分类层。



基于序列标注的方法 – SummaRuNNer 算法

SummaRuNNer 算法逐级对文本进行处理，使用双向 GRU 进行建模。**输入层将句子单词转换为词向量表示。**

- ▶ 单词层在词级别对文本进行处理，双向地依次根据每个单词的词嵌入以及历史隐状态来计算每个单词的隐状态。
- ▶ 利用平均池化方法将句子中所有单词的隐状态融合为句子表示，并作为句子层的输入。
- ▶ 句子层则是在句子级别对文本进行处理，同样使用双向 GRU 进行建模，然后利用所有句子的隐状态通过如下公式计算出文本的整体表示：

$$d = \tanh \left(W_d \frac{1}{N_d} \sum_{j=1}^{N_d} [h_j^f, h_j^b] + b \right)$$

其中， h_j^f, h_j^b 分别是文本第 j 个句子前向 (Forward) 和后向 (Backward) 处理时的隐状态， N_d 是文本中的句子数量， $[\cdot, \cdot]$ 表示将两个向量进行拼接

基于序列标注的方法 – SummaRuNNer 算法

- SummaRuNNer 算法的最后一层是分类层，从第一句开始，结合句子的隐状态以及文本的整体表示，依次判断各个句子是否需要提取出来作为摘要的一部分，最终实现摘要的抽取。具体的分类方法：

$$P(y_j = 1|h_j, s_j, d) = \sigma(W_c h_j + h_j^T W_s d - h_j^T W_r \tanh(s_j) + W_{ap} p_j^a + W_{rp} p_j^r + b)$$

其中， h_j 是通过对 $[h_j^f, h_j^b]$ 进行非线性变换得到的第 j 个句子的嵌入表示， s_j 是摘要在处理第 j 句子时的动态表示：

$$s_j = \sum_{i=1}^{j-1} h_i P(y_i = 1|h_i, s_i, d)$$

$W_c h_j$ 反映句子的信息量， $h_j^T W_s d$ 反映句子对于文章的重要性， $h_j^T W_r \tanh(s_j)$ 反映句子对于已提取部分的冗余程度， $W_{ap} p_j^a$ 和 $W_{rp} p_j^r$ 分别考虑了句子的在文本中的绝对位置 (实际的句子数) 和相对位置 (将文本分割成固定数量的段，句子所属的段数就是相对位置)， P^a 和 P^r 也是需要学习得到的模型参数，分别是句子绝对位置和相对位置的嵌入表示。

基于序列标注的方法 – SummaRuNNer 算法

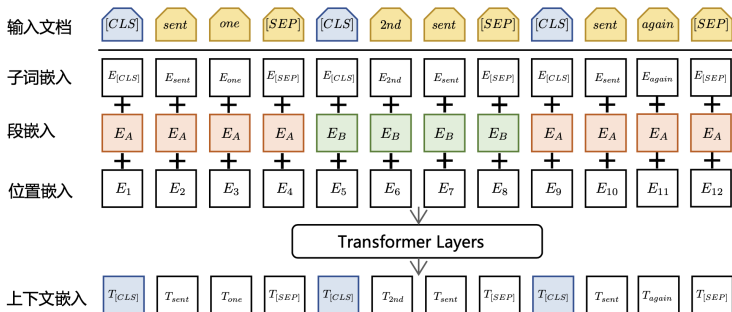
- 根据上述公式得到模型的预测结果后，SummaRuNNer 算法采用如下损失函数，对模型参数进行学习：

$$\mathcal{L}(W, b) = - \sum_{d=1}^N \sum_{j=1}^{N_d} (y_j^d \log P(y_j^d = 1 | h_j^d, s_j^d, d_d) + (1 - y_j^d) \log(1 - P(y_j^d = 1 | h_j^d, s_j^d, d_d)))$$

其中， N 表示训练集中的文档数量， N_d 表示文档 d 中句子的数量。

基于序列标注的方法 - BERTSum-Ext 算法

- ▶ BERTSum-Ext 是一个基于 BERT 编码器的抽取式摘要模型。由于原始的 BERT 是针对字词进行编码，而基于序列标注的抽取式文本摘要是一句操作和分类，在做句子级别的序列标注时，要对输入 BERT 的内容做一定调整，需要进一步计算句子的向量表示。



基于序列标注的方法 – BERTSum-Ext 算法

- ▶ BERTSum-Ext 与 BERT 一样，同样也是使用了 [SEP] 标记来分隔句子，而不同之处在于两者的 [CLS] 标记的含义。
- ▶ 在原始 BERT 中，[CLS] 标记被添加在文本序列的开始，该位置整合了输入序列中的全局信息。
- ▶ 在 BERTSum-Ext 中，[CLS] 标记被添加在每一个句子的开始位置，该句子的有效信息将会被整合到对应 [CLS] 标记位置的字段上。
- ▶ 此外，BERTSum-Ext 会为位置是奇数和偶数的句子分别赋予不同的段嵌入(Segment Embeddings)，即第 1, 2, 3, 4... 句将分别被赋予段嵌入 $E_A, E_B, E_A, E_B \dots$ 。

基于序列标注的方法 – BERTSum-Ext 算法

- ▶ 得到不同位置的 [CLS] 向量 t_i 来表示各个句子，然后将其输入到后续的 Transformer 模块中进行更新:

$$\tilde{h}^l = \text{LayerNormalization}(h^{l-1} + \text{MultiHeadAttention}(h^{l-1}))$$

$$h^l = \text{LayerNormalization}(\tilde{h}^l + \text{FFN}(\tilde{h}^l))$$

使用顶层输出的句子嵌入作为最终输入分类器的表示。分类器可以为每个句子表示计算并预测二元分类结果，表示各个句子是否需要被提取出来作为摘要的一部分:

$$\hat{y}_i = \sigma(W_o h_i^L + b_o)$$

- ▶ 使用模型预测值与参考标签之间的交叉熵作为损失函数，就可以对模型进行训练。在测试阶段，选择预测概率高于一定阈值的句子，作为最终抽取得到的摘要。

基于序列标注的方法 – 基于 HMM 的生成方法

- ▶ 基于序列标注的方法需要大规模的有标注训练语料，需要人工确定文章中每个句子是否应被抽取，因此文本摘要的人工标注的代价十分高昂。但是，我们可以从互联网上收集到很多，包含人工撰写的摘要以及相应文章的文本-摘要对，而且人们在构造摘要的过程中通常都会使用到文档中的原词甚至原句。因此，研究人员们开始思考**是否可以使用现有的文本-摘要对，来进行分类器的训练**。一种思路就是为摘要中的句子在正文中找出与其相似度最高的句子，并用它来作为对应部分的摘要，即**对齐 (Alignment) 算法**。
- ▶ 通过对齐算法，能够产生大规模的文档-摘要对，从而可以进行有监督的训练。简单的对齐算法有**计算正文和摘要句子的最长公共子序列或编辑距离**等，这些方法也可以结合使用。

基于序列标注的方法 – 基于 HMM 的生成方法

- ▶ 该算法的思路是将摘要中的句子分解成字或词 (后续简称为词)，然后从句首开始往后依次找到这些词在正文中的出现位置，进而就可以找到相应的句子来构成近似甚至相同的摘要。由于同一个词可能在文章中多次出现，所以摘要中的单个词就存在多个可能的出现位置，需要去判断其中哪一个是正确的。

例如，在一篇文章的摘要中出现了 “the communication subcommittee of”，而 the, communication, subcommittee, of 分别在正文中出现了 44, 1, 2, 22 次，也就意味着有 $44 \times 1 \times 2 \times 22$ 共 1936 种可能的位置组合。

- ▶ 在该隐马尔可夫模型中，词每个可能的出现位置都是可以观测到的，并且这些位置都各自对应着一个状态 (S, W) ，它表示这个词出现在正文第 S 句话的第 W 个位置。假设词 I_{i+1} 和词 I_i 是相邻的两个单词，可以使用 $P(I_{i+1} = (S_2, W_2) | I_i = (S_1, W_1))$ 表示当词 I_i 出现在 (S_1, W_1) 位置时， I_{i+1} 出现在 (S_2, W_2) 位置的条件概率。

基于序列标注的方法 – 基于 HMM 的生成方法

- ▶ 该算法使用一种启发式赋值方式，按照当前处理词 I_{i+1} 和上一个处理的词 I_i 的相对位置分成 6 种不同的情况来梯度赋值：

- (1) $S_1 = S_2, W_1 = W_2 - 1$ ，即 I_{i+1} 和 I_i 出现在正文的同一个句子中，且 I_i 刚好在 I_{i+1} 的前一个位置。
- (2) $S_1 = S_2, W_1 < W_2 - 1$ ，即 I_{i+1} 和 I_i 出现在正文的同一个句子中，且 I_i 在 I_{i+1} 的前方，但二者不相邻。
- (3) $S_1 = S_2, W_1 > W_2$ ，即 I_{i+1} 和 I_i 出现在正文的同一个句子中，但 I_i 在 I_{i+1} 的后方。
- (4) $S_2 - \text{CONST} < S_1 < S_2$ ，即在正文中， I_{i+1} 出现在 I_i 所属句子的后方，且二者之间不会超过 CONST 个句子。
- (5) $S_2 < S_1 < S_2 + \text{CONST}$ ，即在正文中， I_{i+1} 出现在 I_i 所属句子的前方，且二者之间不会超过 CONST 个句子。
- (6) $S_2 - S_1 \geq \text{CONST}$ ，即在正文中， I_{i+1} 和 I_i 的位置间隔了 CONST 以上个句子。

其中， CONST 是一个较小的正数。上述六种情况，其出现的可能性是递减的，确切的概率值由人工设定，可以通过多次实验来选择较优值，在处理不同类型的文章时还可以做一些适应性调整。

基于序列标注的方法 – 基于 HMM 的生成方法

- ▶ 对一个给定的序列，为了找到最优的序列就需要使 $P(I_1, I_2, \dots, I_N)$ 最大化。这里假设摘要中的每个词在正文中的实际位置都仅受其前一个词 I_i 在正文中的实际位置的影响。可以将上述联合概率转换为如下条件概率乘积：

$$P(I_1, I_2, \dots, I_n) = \prod_{i=0}^{N-1} P(I_{i+1} | I_i)$$

根据人工设定的条件概率，就可以计算出每种位置序列的概率，用概率最大的位置序列去找相应句子组合成摘要就可以取得不错的效果。需要注意的是有些摘要中的词不一定会出现在原文中（例如，衔接词），对这些词进行特殊化处理，比如用 $S = -1$ 来表示该词没有在正文中出现，然后在计算时跳过该词。整个求解过程可以采用 Viterbi 算法完成。

11.3 生成式文本摘要

生成式文本摘要

- ▶ 抽取式文本摘要所获得的结果都来源于原始文本，因此得到摘要文本中每个单句的语义准确性和语法正确性都可以得到很好的保证。但是由于原始的句子来源于不同的段落甚至篇章，所生成的摘要中句子之间的连贯性则很难保证，造成摘要整体的可读性较差。
- ▶ 生成式文本摘要则可以产生原始文档中不存在的新文本。通常生成式文本摘要算法需要首先构建原始文档的抽象语义表示（编码过程），然后使用此语义表示创建出接近人类表达方式的摘要（解码过程）。该类算法通常能产生更精简凝练并且更具有连贯性的摘要。
- ▶ 语义表示过程也可应用于图像和视频，因此生成式摘要方法也可用来处理多模态文本摘要。
- ▶ 生成式摘要比抽取式摘要更具挑战性，它不仅涉及自然语言生成，还涉及依赖领域知识对原始文档的深入理解。

生成式文本摘要

- ▶ 在 2014 年，Google Brain 团队首次提出了序列到序列生成结构（Sequence-to-Sequence, Seq2Seq）来进行文本生成任务。序列到序列生成结构主要由编码器（Encoder）和解码器（Decoder）组成，其中编码器负责将输入文本中所需的语义信息编码成向量，解码器则负责从编码向量中解码出语义信息并生成特定文本。
- ▶ 对应到生成式文本摘要，可以将原始文档输入编码器，编码器进行文档的理解并将其中的关键信息进行编码，解码器则将关键信息进行解码并生成对应的文本摘要。

序列到序列生成文本摘要 – 基于预训练的生成式方法

- ▶ BERTSum-Abs 是结合了预训练语言模型的 Seq2Seq 摘要生成方法。它的基本框架也是基于 Transformer 结构，其区别在于，编码器是预训练语言模型（如 BERT），而解码器则采用参数随机初始化的多层 Transformer 神经网络结构。这样的设计虽然可以很好的利用了预训练语言模型，能够对文本内容进行很好的编码。但是会导致编码器和解码器之间存在着参数状态不匹配的问题，即编码器的模型参数是经过预先训练的，而解码器的模型参数是随机初始化的。由于在相同的优化策略下，解码器相比于编码器更难收敛，这可能会使训练阶段的微调过程不稳定。
- ▶ 为了解决这一问题，BERTSum-Abs 设计了一个新的微调范式，用不同的优化策略对编码器和解码器进行微调。BERTSum-Abs 采用的方法是将解码器的学习率设置更大，使其参数更新的幅度更大，逐渐缩小两者不匹配的参数状态。

序列到序列生成文本摘要 – 基于预训练的生成式方法

- ▶ 具体来说，BERTSum-Abs 对编码器和解码器分别使用了不同的 Adam 优化器参数，它们具有不同的预热步数（Warm Up Step）和学习率：

$$\text{lr}_\epsilon = \tilde{\text{lr}}_\epsilon \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}_\epsilon^{-1.5})$$

$$\text{lr}_D = \tilde{\text{lr}}_D \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}_D^{-1.5})$$

其中 lr_ϵ 为编码器学习率， lr_D 为解码器学习率。

通过上述参数控制，使得预训练的编码器以更小的学习率和更平滑的衰减速率进行微调。这样，当解码器参数变得稳定时，编码器就可以得到更精确的梯度进行参数更新。

- ▶ 此外，BERTSum-Abs 还提出了一种两阶段的微调方法，首先在抽取式摘要 (BERTSum-Ext) 上微调编码器，第二阶段再进行生成式摘要的微调。这两个阶段使用的是同一个编码器，从而可以让模型利用这两种模式之间共享的编码信息。

序列到序列生成文本摘要 – 基于预训练的生成式方法

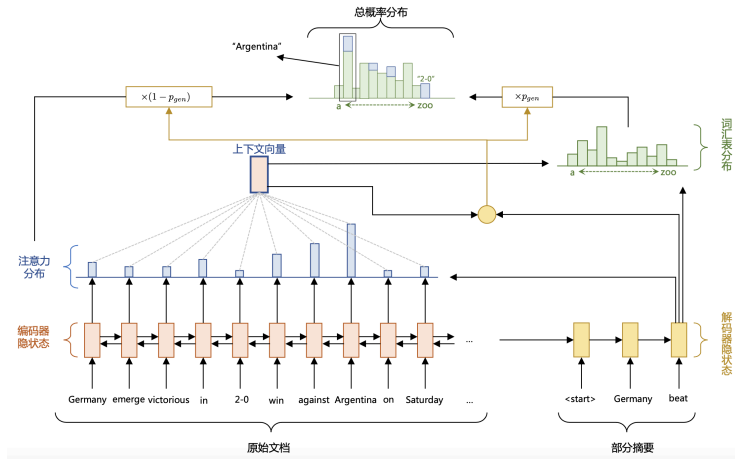
- ▶ 上述方法中，BERTSum-Abs 使用的是预训练编码器，并结合参数随机初始化的解码器进行微调。这种解决方案主要是针对早期的预训练语言模型 (BERT 等)，它们侧重于文本的理解和编码。
- ▶ 随着预训练语言模型的不断发展，陆续出现了对编码器和解码器联合预训练的解决方案，其可以直接被应用在适合序列到序列生成架构的文本生成任务中，并在生成式文本摘要任务中展现了更好的性能，比如 BART 等。由于 BART 预训练方法本身是一个去噪自编码器，与摘要任务的形式有一定的关联性，即去除文档中的冗余和无关的信息，只保留关键信息。因为这种上游预训练形式与下游任务相关的特性，BART 在生成式文本摘要中可以获得更好的效果。

序列到序列生成文本摘要 – 基于复制与覆盖机制

- ▶ 由于生成式文本摘要方法会对原始文档的内容进行转换和整合，所以如何保证摘要中的关键信息不重复不遗漏是非常重要的问题。一方面，原始文档可能会包含重要的生僻词甚至未登录词，而这些词在解码过程中被生成的概率极低，就会造成关键信息的遗漏问题。另一方面，一些在原始文档中反复出现的关键信息也有可能被解码器生成多次，造成摘要冗余和不通顺的问题。
- ▶ 为了解决以上问题，针对生成式文本摘要的复制与覆盖机制提出了 PGNet。PGNet 使用了指针解码器，通过指针从原始文本中直接复制单词，同时也可以从词表产生原文档中未出现的新单词。此外，PGNet 还使用了覆盖向量来跟踪和控制摘要对原文档的内容覆盖范围，从而能减少冗余现象。

序列到序列生成文本摘要 - 基于复制与覆盖机制

PGNET 的神经网络结构



序列到序列生成文本摘要 – 基于复制与覆盖机制

- PGNet 使用了 RNN 文档编码器，将原始文档的所有单词输入一个单层双向 LSTM 模型中，得到编码器隐状态 h_i 。解码器是一个单层单向 LSTM，在每个时间步 t 上，接收前一个单词的嵌入表示，并得到解码器状态 s_t 。之后，PGNet 使用注意力分布 a^t 来衡量第 t 个解码时间步上每个单词的重要性：

$$e_i^t = v^T \tanh(W_h h_i + W_s s_t + b_{\text{attn}})$$
$$a^t = \text{softmax}(e^t)$$

PGNet 利用注意力分布来产生编码器隐状态的加权和，称为上下文向量 h_t^* ：

$$h_t^* = \sum_i \alpha_i^t h_i$$

上下文向量 h_t^* 可以看作是在每个时间步，解码器从原文档中读取的全局内容表示，它与解码器状态 s_t 连接，并通过两个线性层来产生词汇表分布 P_{vocab} ：

$$P_{\text{vocab}} = \text{Softmax}(V'(V[s_t, h_t^*] + b) + b')$$

序列到序列生成文本摘要 – 基于复制与覆盖机制

- ▶ 除了从词表单词的概率分布中生成单词，PGNet 还允许通过指针从原文档中复制单词，从而有机会将文档中的生僻词和未登录词写入摘要。模型在 t 时刻需要确定解码器应当从词表中生成单词，还是从原文档中复制单词。
- ▶ PGNet 设计了一个选择器 $p_{\text{gen}} \in (0, 1)$ ，以 $0 - 1$ 之间的概率来判断此时间步的操作，应当是从 P_{vocab} 中采样生成一个单词，还是通过从注意力分布 a^t 中采样来复制原文档中的一个单词。时间步 t 的二元选择概率 p_{gen} 基于全局语义向量 h_t^* 、解码器状态 s_t 和解码器输入 x_t ，通过如下公式计算：

$$p_{\text{gen}} = \sigma \left(w_{h*}^T h_t^* + w_s^T s_t + w_x^T x_t + b_{\text{ptr}} \right)$$

序列到序列生成文本摘要 – 基于复制与覆盖机制

- ▶ 最终，可以得到词表与原始文档中单词的总概率分布，即结合了词表分布和注意力分布来采样单词。这也可看作是一种扩展的词表，包含了原词表和原始文档中所有出现单词的联合词表。在扩展词表上的概率分布可计算如下：

$$P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i:w_i=w} \alpha_i^t$$

如果单词 w 是一个未登录词，那么 $P_{\text{vocab}}(w)$ 为 0。类似地，如果 w 没有出现在原始文档中，那么 $\sum_{i:w_i=w} \alpha_i^t$ 为零。产生未登录词的能力是 PGNet 模型的主要优势之一。

序列到序列生成文本摘要 – 基于复制与覆盖机制

- ▶ 除了解决生僻词和未登录词的遗漏问题，PGNet 还使用了一个简单的方案解决摘要中单词的重复生成问题。单词的重复生成问题是 Seq2Seq 模型的常见问题，在生成多句文本时尤其明显，一个简单的思路是记录所有已经生成的单词，并且避免在后续的解码时间步生成这些词。PGNet 中的覆盖机制也是基于这一思想，维护一个覆盖向量 c_t ，它是解码器的所有历史时间步的注意力分布的和：

$$c^t = \sum_{t'=0}^{t-1} a^{t'}$$

直观来看， c_t 是基于原始文档单词的一种非归一化的分布，它代表了这些单词到 t 时间步为止被注意力分布覆盖的程度。 c_t 一般可以初始化为一个全 0 向量，因为在第一个时间步中，注意力分布还没有覆盖原始文档。

序列到序列生成文本摘要 – 基于复制与覆盖机制

- ▶ PGNet 将覆盖向量作为注意力机制的额外输入作为一种控制冗余的信号。

$$e_i^t = v^T \tanh(W_h h_i + W_s s_t + w_c c_i^t + b_{\text{attn}})$$

通过上述公式，PGNet 确保了当前的注意力分布是通过之前时间步的注意力分布得到的，从而可以避免注意力分布在相同位置的单词上赋予过高的权重，避免产生重复的文本。

- ▶ PGNet 还设计了额外的损失函数 covloss_t ，以惩罚反复关注同一单词位置的情况：

$$\text{covloss}_t = \sum_i \min(a_i^t, c_i^t)$$

上述公式惩罚了 t 时间步前每个注意力分布和覆盖向量之间的重叠，从而防止了注意力分布的重复。

- ▶ 最后，将上述损失函数通过超参数 λ 加权添加到主损失函数中，得到一个新的复合损失函数：

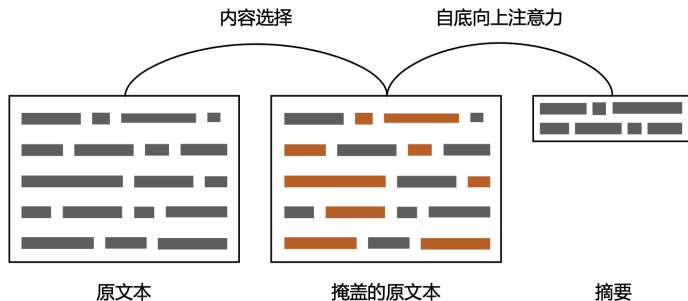
$$\text{loss}_t = -\log P(w_t^*) + \lambda \sum_i \min(a_i^t, c_i^t)$$

抽取与生成结合式文本摘要

- ▶ 抽取式文本摘要和生成式文本摘要都存在着各自的优势，于是研究员人们也尝试将抽取式方法和生成式方法结合，并形成一类新的方法。
- ▶ 这一类方法的算法流程为：首先对原文档的内容进行抽取，得到初步的摘要内容；再将抽取的结果输入生成器，经过优化得到最终的摘要。
- ▶ 我们可以根据第一步中抽取内容的粒度（如字词、句子）对这一类方法进行区分：词粒度抽取与生成结合方法、句子粒度抽取与生成结合方法。

抽取与生成结合式文本摘要 – 词粒度抽取与生成结合

- **Bottom-Up**是一种以字词为抽取粒度的方法，其思路是将摘要生成分成如下图所示两个主要步骤：
- (1) 通过自底向上的注意力机制从文本中抽取可以作为摘要的单词；
 - (2) 用第一步抽取出的单词辅助生成摘要。



抽取与生成结合式文本摘要 – 词粒度抽取与生成结合

- 对于第一步抽取单词，可以直接参考基于序列标注的抽取式方法，让文档中的每个单词都拥有一个二元标签 (1 和 0，用于表示该词语能否作为摘要的一部分) 以及一个嵌入表示用于完成相应的序列标注。初始嵌入表示 e_i 由两个部分 e_i^w 和 e_i^c 拼接而成， e_i^w 是预训练模型 (如 BERT) 学习到的词嵌入， e_i^c 则是用一个双层双向长短期记忆网络计算得到：

$$e_i^c = \gamma \times \sum_{l=0}^2 s_j \times h_i^{(l)}$$

在计算出 e_i 后，还需要将其输入到另一个单层双向长短期记忆网络中，以此计算出每个单词的最终嵌入 h_i ，然后就可以通过 h_i 计算出各单词能用于生成摘要的概率 q_i ：

$$q_i = \sigma(Wh_i + b)$$

抽取与生成结合式文本摘要 - 词粒度抽取与生成结合

- 在步骤二中，摘要生成可以通过一个 Seq2Seq 模型来实现。在每个时间步 j ，模型可以选择从词表中生成一个单词或者直接从上一步抽取出的单词中进行复制：

$$p(y_j|y_{1:j-1}, x) = p(z_j = 1|y_{1:j-1}, x) \times p(y_j|z_j = 1, y_{1:j-1}, x) \\ + p(z_j = 0|y_{1:j-1}, x) \times p(y_j|z_j = 0, y_{1:j-1}, x)$$

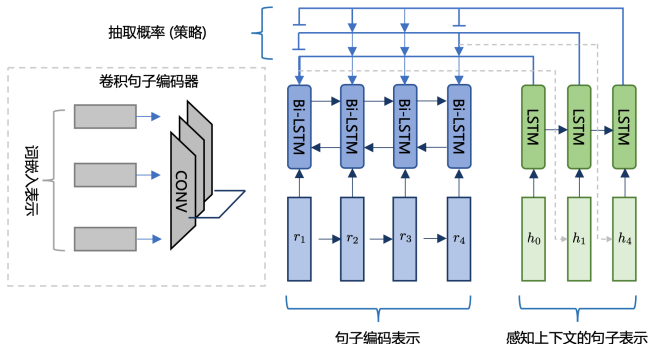
其中的 $z_j = 1$ 表示当前时间步需要从词表中生成一个字词， $z_j = 0$ 表示当前时间步直接从抽取单词集合中复制一个单词。在复制抽取的单词时，可以引入注意力机制来提升性能，即把分布 $p(y_j|z_j = 0, y_{1:j-1}, x)$ 替换成：

$$p(\tilde{a}_j^i|x, y_{1:j-1}) = \begin{cases} p(a_j^i|x, y_{1:j-1}) & q_i > \epsilon \\ 0 & \text{其他} \end{cases}$$

其中 a_j^i 表示时间步 j 时单词 w_i 的注意力权重。

抽取与生成结合式文本摘要 - 句子粒度抽取与生成结合

- FastRL 是一种以句子为粒度的抽取与生成方法，其生成摘要的过程也可以大致分为两个步骤：(1) 抽取器提取文章中重要的句子；(2) 生成器对第一步提取出来的句子进行压缩和改述，并生成一段连贯的摘要。



抽取与生成结合式文本摘要 - 句子粒度抽取与生成结合

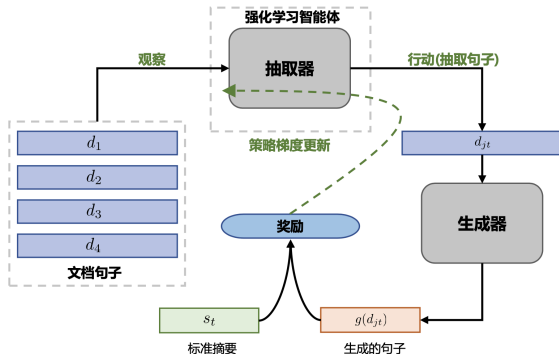
- ▶ 与 Bottom-Up 算法的不同之处在于，FastRL 使用了强化学习的方法对抽取和生成的两阶段模型进行了联合训练，使用了生成模型的摘要结果作为奖励函数来优化抽取模型。
- ▶ 抽取器共由三个部分组成。第一个部分是卷积神经网络，通过卷积和池化的方式为文本中的每个句子计算出一个表示 $r_j, j = 1, 2, \dots, n$ 。第二个部分是双向长短期记忆网络，它以 r_j 为输入进一步为文本中的每个句子计算出结合全局信息的表示 $h_j, j = 1, \dots, n$ 。第三部分是单向长短期记忆网络，负责完成最终的句子选择，这个部分引入了两跳注意力机制 (2-Hop Attention Mechanism)，首先计算 h_j 对应的包含上下文信息的表示 e_t ，再用其计算每个句子被抽取的概率 $p(j_t | j_1, \dots, j_{t-1})$ ：

$$u_j^t = \begin{cases} v_p^T \tanh(W_{p1} h_j + W_{p2} e_t) & \text{if } j_t \neq j_k \quad \forall k < t \\ -\infty & \text{otherwise} \end{cases}$$

$$P(j_t | j_1, \dots, j_{t-1}) = \text{softmax}(u^t)$$

抽取与生成结合式文本摘要 - 句子粒度抽取与生成结合

- 因为需要同时训练抽取器和生成器，但抽取器和生成器之间无法直接传递梯度信息，所以FastRL 使用了强化学习中的策略梯度来协同训练抽取器和生成器：用生成器生成的结果与真实摘要进行比较，然后将比较结果作为反馈来更新抽取器的参数。下图给出了强化学习的过程，其中的 $d_1, d_2, \dots, d_n, s_t$ 是文本正文以及摘要中的句子。



11.4 文本摘要的评测

文本摘要的评测

- ▶ 与文本分类、命名实体识别等其他自然语言处理任务的评测相比，由于关键信息的选取和文摘的表述没有统一的标准答案，因此文本摘要的人工评测和自动评测都困难很多。与机器翻译、对话系统等面向内容生成（Content Generation）的任务一样，文本摘要也有多维度的评价系统，包括人工评测方案和自动评测指标。
- ▶ 文本摘要的评测按照与任务的相关性可以分为两类：
 - a) **内在评价**（Intrinsic Evaluation）与摘要任务相关，它通过直接分析摘要的质量来评价摘要系统，一般从总体和细分维度两个粒度进行评估。其中，总体评估是指从总体上评价摘要的质量，并给出一个综合分数。细分维度一般可以从“与参考标准的一致性”和“类人性”两个角度来考虑，多方面地对摘要的质量进行评价。
 - b) **外在评价**（Extrinsic Evaluation）则是一种间接的评价方法，与系统的功能对应，将摘要应用于下游任务或者实际应用中，根据摘要在这些任务和系统中产生的实际效果来间接评估摘要的质量。这里我们仅讨论内部评价方法。

文本摘要的评测

在评测文本摘要的质量时会关注以下5个细分维度

- ▶ (1) 信息量 (Informativeness), 即摘要的内容含量, 它可以衡量一段文本所提供的新信息的程度。文献 [615] 对信息量作出了直观上的定义: 阅读者一般都拥有背景知识和常识, 如果一段摘要能使其获得新信息或者产生认知上的变化, 则可认为此摘要具有丰富的信息量。
- ▶ (2) 非冗余性 (Non-redundancy), 它体现了摘要精简的特性, 即不能反复使用重复或相似的文本描述同一个关键点。
- ▶ (3) 流畅度 (Fluency), 它包含了摘要结构的连贯性和语法的正确性两个方面。
- ▶ (4) 忠实度 (Faithfulness), 也称为相关性 (Relevance), 即摘要内容是否忠于原文。作为原文的子集, 摘要不能包含凭空产生的信息, 也不能包含与原文和事实相悖的内容。
- ▶ (5) 聚焦程度 (Focus), 也称为显著性 (Saliency), 它衡量了摘要包含关键信息的程度。除了主要信息, 原文档往往具有大量的细节描述和补充内容等次要信息。通过聚焦程度, 可以评估一个摘要系统的甄别并捕捉原文主要信息的能力。

人工评测

人工评测按照执行方式一般分为两类：

- ▶ **逐点评估 (Point-wise)**：评估者对系统产生的每一个结果按照预定义的维度进行评估评分。一个常见的方案是李克特五点量表 (Likert Scale)。给定原始文档和模型输出的摘要，评估者会按照 1-5 分对摘要某一方面的评测维度进行评分。假设评测维度是流畅度，则 1-5 的分值依次对应：非常不流畅、不流畅、一般、流畅、非常流畅五种程度。但是，逐点评估具有很强的主观性，导致评估者之间的偏差较大，一致性很低。
- ▶ **逐对评估 (Pair-wise)**：给定相同的原始文档和两个不同系统的输出摘要 A 和 B，评估者需要判断 A 与 B 相比哪个更好。与逐点评估的多选项评分相比，逐对评估采用两两比较的方式，降低了评估难度，可以提高评测结果的一致性。但是，如果存在多个需要评测的摘要系统，总评估次数会随着系统数目的增加而呈 $O(N^2)$ 的复杂度上升，成本较高。

人工评测

- ▶ 人工评测存在主观性，包括摘要任务本身的主观性和评估者自身的主观性。为了尽可能消除人工评测的主观性偏差，一般会让多个评估者对同一条数据进行独立重复评分。因此，衡量多个评估者之间的评测一致性是一个重要的过程。一致性不仅可以体现人工评测质量的高低，还能反映评测任务的难易程度。Fleiss 卡帕系数也称 Kappa 系数，是度量多个系统一致性的方法，也经常被用于计算人工评测的一致性。
- ▶ Fleiss 卡帕系数在不同的区间显示不同的一致性程度。系数小于 0 时表示没有一致性，0~0.20 表示轻微 (Slight) 一致，0.21~0.40 表示一般 (Fair) 一致，0.41~0.60 表示中等 (Moderate) 一致，0.61~0.80 表示显著 (Substantial) 一致，0.81~1.00 表示完美 (Perfect) 一致。

自动评测

- ▶ 人工评测虽然可以提供丰富的信息，但是大规模的人工评测耗时长、工作量大、成本高。自动评测则可以提供高效、低成本、一致的评测，在模型开发过程中这些特点都受到研究人员青睐。随着评测技术的发展，自动评价结果也具有了更好的指导意义。
- ▶ ROUGE (Recall-Oriented Understudy for Gisting Evaluation)，称为面向召回率的要点评估，也是文本摘要中最常用的自动评价指标之一。ROUGE 与机器翻译的评价指标 BLEU 的类似，能根据机器生成的候选摘要和标准摘要（参考答案）之间词级别的匹配来自动为候选摘要评分。
- ▶ ROUGE 包含一系列变种，其中应用最广泛的是 ROUGE-N，它统计了 n-gram 词组的召回率，通过比较标准摘要和候选摘要来计算 n-gram 的结果。

自动评测

- ▶ ROUGE 是基于候选文本与参考文本之间的精准匹配来评价文本的质量。但是在一些情况下，不同的词语或短语可以表达同一种语义。
例如，“立刻”和“马上”，两者的含义相同，但是基于 n-gram 的匹配就会失效。
一个常用的解决方案为基于嵌入表示的度量 (Embedding-Based Metrics)，使用分布式词嵌入来计算词语之间的相似度，具体来说包括静态词向量和上下文相关的词向量两种方案。

11.5 文本摘要语料库

文本摘要语料库

- ▶ 如前所述文本摘要被广泛应用于多种场景中，任务类型包括单文档摘要、多文档摘要、对话摘要、跨语言文本摘要和多模态文本摘要等多种类型。
- ▶ 单文档摘要语料库：
 - a) CNN/DailyMail: 该数据集是被广泛关注和研究的短文本摘要数据集，它包含了 311,672 个新闻/摘要对，新闻数据来源是美国有线电视新闻网和《每日邮报》。新闻平均长度为 766 个词 (29.74 个句子)，摘要的平均长度为 53 个词 (3.72 个句子)。
 - b) LCSTS: 该数据集是常用的中文短文本摘要数据集，包含 240 万个新闻/摘要对，数据来源为微博认证的官方新闻。
 - c) Arxiv/PubMed: 这两个数据集是被广泛应用的长文本数据集，分别来源于 arXiv 和 PubMed 的学术网站上的论文和摘要。ArXiv 数据集包含 21.5 万个论文/摘要对，论文平均长度为 4938 个词，摘要平均长度为 220 个词。PubMed 数据集包含 13.3 万个论文/摘要对，论文平均长度为 3016 个词，摘要平均长度为 203 个词。

文本摘要语料库

► 多文档摘要语料库：

a) Multi-News: 该数据集是大规模的多文档摘要数据集，场景是新闻文档，数据来源是 Newser 网站。此网站上的新闻稿会引用多个新闻源，这些新闻源被归为输入文档集合。

Multi-News 共包含 56216 个文档集合/摘要对，文档集合的平均长度为 2103 个词，摘要的平均长度为 264 个词。

b) WikiSum: 该数据集是基于英文维基百科的多文档摘要数据集。在每个实例中，输入由维基百科主题（文章标题）和非维基百科参考文档的集合组成，目标是生成维基百科文章的精简文本。数据集以 8:1:1 的比例被划分为训练集、验证集和测试集，分别包含 1,865,750、233,252 和 232,998 个样本。

本章小结

- ▶ 文本摘要是一种利用算法自动实现文本分析、内容提炼并生成摘要的技术。尽管基于深度学习尤其是预训练技术的文本摘要已经取得了显著的进展，但在真实场景大规模应用文本摘要仍面临着诸多挑战。

首先是数据资源的问题，取得高质量的摘要数据往往成本高昂，有时候我们必须面临低资源情况下训练数据缺乏的问题。

其次是摘要的场景丰富多样，不仅包括文本的类别（新闻、评论、学术论文等），还包括多模态数据（对话、图像、视频等），这些场景领域差异大，导致训练好的摘要模型难以迁移。

最后，文本摘要的评估也是亟待解决的难题，设计合理的高效的评估方法能促进模型的迭代，但其具有很大的挑战性。

本章小结

第十一章 文本摘要

11.1 文本摘要概述

11.2 抽取式文本摘要

11.3 生成式文本摘要

11.4 文本摘要的评测

11.5 文本摘要语料库